

- 学习目标
- 这篇讲义怎样读
- 一、同一个坑摔两次：缺少记忆的 Agent 长什么样
 - 记忆系统有四个动作，RAG 只覆盖其中一个
- 二、四层记忆理论：先分清放在哪里
- 三、四层记忆怎么抽取：来源、压缩和输出
 - 四层记忆的抽取方式
 - 先只写 raw 层，看"没有自进化整理"的召回长什么样
 - 分步整理：每一层都有暂存产物
 - 语义层输出：同义合并与三条晋升通道
- 四、心跳整理：定时触发第三节的抽取管线
 - 管线：一次心跳做四件事
 - 心跳本体：扫描 checkpoint，触发文件整理步骤
 - 心跳的安全边界：自动整理不能变成自动污染
- 五、新任务唤起：从文件记忆构建 working memory
 - 召回的四步
 - 文件产物：让进化后的记忆能被新任务找到
- 六、综合案例：文件记忆接入 Workspace 后执行新任务
 - 上海亲子游的记忆增强对比
- 七、工程边界：谁负责什么，谁不负责什么
- 总结
- 资料来源

《搭建类似Hermes Agent的自进化长期记忆》

从 Hermes Agent 的长期记忆结构出发，用基础文件系统、TriggerFlow、agent direct request 和最后的 Workspace 综合案例，搭一个可解释的自进化闭环。

学习目标

概念导论的责任地图里有一块"会话、状态与记忆"。其中会话内的状态在窗口里就能维持，真正难的是另一半：任务结束之后，哪些经历值得留下？留下之后怎样被修正、被

遗忘？下一次任务开始时，又怎样把该带的带回来？ 本文要搭一套类似 Hermes Agent 的长期记忆自进化闭环。

读完本文，应当能够：

- 说明长期记忆要解决的问题，以及它和 RAG 的边界在哪里；
- 把四层记忆和 Hermes Agent 的 `MEMORY.md` / `USER.md`、`session_search`、Memory Provider 对上，说清每层放什么、从哪里来、压缩成什么输出；
- 用 agent direct request 抽取会话摘要、候选经验和稳定规则；
- 把心跳理解成定时触发整理管线的后台任务，而不是另一套记忆机制；
- 用文件产物构建 working memory，并在最后综合案例里接入 Workspace 验证行为变化。

这篇讲义怎样读

整篇讲义沿一条线推进：先看到缺记忆的痛，再建立四层记忆理论，接着讲每一层从哪里抽取、怎样压缩、输出成什么，然后再把这套抽取管线放到心跳里定时触发，最后用新任务唤起和综合案例验证效果。

有一条分工原则贯穿全文：理解自然语言的判断交给模型（哪句话值得记、两条候选是不是同一件事）；可核对的结构化判断留在代码（工具返回的 `status`、跨会话出现次数、证据链接、`checkpoint`）。前半段只用文件系统保存中间产物，Agently 只负责 agent direct request 和 TriggerFlow；Workspace 在最后的综合案例里才出现，负责工程集成。

运行环境：课程 3.10 环境 + 线上 Agently==4.1.3.8 + DeepSeek 模型接口。向量库不是必需组件，只作为可选扩展

(`extensions/chromadb_hybrid_retrieval.py`)。

一、同一个坑摔两次：缺少记忆的 Agent 长什么样

小李做了一个旅行规划 Agent。6 月中旬，用户让它排北京三日亲子游：初版把故宫和环球影城排进了同一天，路线检查工具报了失败；修正成"同区域聚合、远距离大项单独占一整天"之后检查通过。用户临走前还特意交代了一句："以后给我这种结果，把检查结论一起附上。"

一周后，同一个用户来排西安两日游。Agent 把兵马俑和回民街塞进了同一天——兵马俑在临潼区，距市区约 40 公里，这和上周北京那次是同一类错误。用户的"附检查结论"交代，它也像没听过一样。

这不是模型能力问题。会话结束时上下文被丢弃，上一次的失败证据、修正方案、用户交代，全都留在了那段再也不会被读到的聊天记录里。要解决它，第一反应往往是"上 RAG"——但先看一下 RAG 和记忆系统各自覆盖什么。

记忆系统有四个动作，RAG 只覆盖其中一个

RAG 回答的问题是"从一批资料里找相关片段"。资料本身是静态的：不管任务成败，它都不会变。记忆系统面对的是不断产生的经历，至少要回答四个问题：

动作	要回答的问题	常见工程实现
ingestion 写入	什么经历进入记忆系统	事件流、工具结果、用户偏好、决策记录
revision 修正	旧记忆怎样被改写	摘要重写、同义合并、证据重链
forgetting 遗忘	什么不再进入候选上下文	过期、降权、归档、删除
retrieval 召回	新任务取回哪些记忆	文本检索、规则过滤、上下文打包



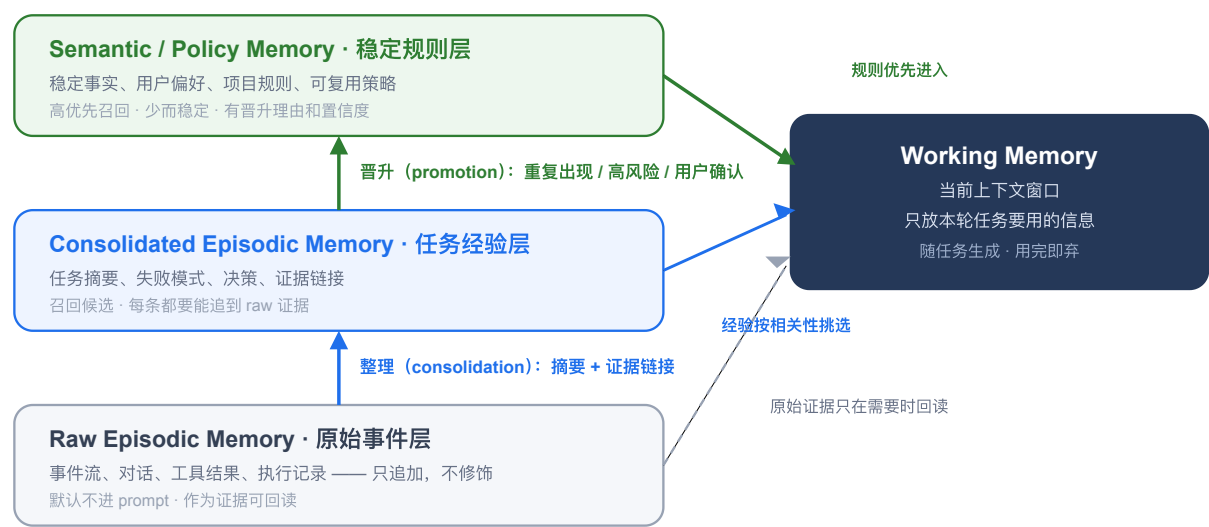
图 1：RAG 只对应"召回"一格；另外三个动作决定"这条记忆值不值得信"。

说白了：数据库负责"存进去还能读出来"；记忆系统还要负责"这条记录在未来该不该影响行为"。后者是治理问题，不是存储问题。

二、四层记忆理论：先分清放在哪里

直接写代码之前，先把长期记忆分层。CoALA（Cognitive Architectures for Language Agents）把语言 Agent 的记忆分成 working、episodic、semantic、procedural 四类：工作记忆、情景记忆、语义记忆、程序记忆。本文把它落到工程上，分成四层，其中 episodic 按治理需要拆成"原始"和"整理后"两层，procedural 收窄为可召回的规则：

四层记忆：左边是生产链，右边是使用方



分层的意义不是名称好看，而是每一层的治理方式不同：进 prompt 的默认权限、修正方式、清理策略都不一样

图 2：左边是生产链——原始事件经整理晋升为稳定规则；右边是使用方——任务按需召回。

层	放什么	默认是否进 prompt	文件产物
Working Memory	当前任务必要信息	是	task_brief.json
Raw Episodic	原始事件、工具输出、对话	否	raw_events.jsonl
Consolidated Episodic	任务摘要、决策、失败原因	候选	consolidated_sessions.jsonl

层	放什么	默认是否进 prompt	文件产物
Semantic / Policy	稳定规则、偏好、长期事实	高优先候选	<code>semantic_rules.jsonl</code>

分层不是为了名称好看，而是为了三件事：只把少量稳定信息放进上下文；需要证据时能回读原始记录；记忆变旧或冲突时能定位并修正。每一层的治理属性都不一样——raw 层只追加、不修饰，semantic 层少而稳定、每条都要有晋升理由。

分层治理也不等于堆数据库。Hermes Agent 的实际结构很值得参考，因为它把长期记忆拆得很克制：小而常驻的稳定记忆、可检索的完整会话历史、以及可选外部 Memory Provider。

证据 · Hermes Agent：小稳定层 + 大历史层 + 外部附加层

来源 Nous Research, Hermes Agent 文档: *Persistent Memory* 与 *Memory Providers*

Hermes Agent 实际结构	本文四层里的位置	本文的落地方式
<code>MEMORY.md</code> / <code>USER.md</code>	Semantic / Policy 的稳定规则与用户偏好	<code>semantic_rules.jsonl</code> ，规则少而稳定，带证据和晋升理由
SQLite + <code>session_search</code>	Raw Episodic 的完整事件流	<code>raw_events.jsonl</code> ，原始对话和工具结果只追加
会话结束抽取 / 后台 review	Raw → Consolidated → Semantic 的整理动作	心跳定时触发文件整理步骤和 checkpoint
Memory Provider	语义检索与外部记忆扩展	可选的向量库扩展（ChromaDB / LanceDB），不作为主存储
Skills 自改进	Procedural memory	收窄为 Semantic / Policy 层的可召回规则；技能生成与修补不属于记忆管线

含义：本文不是照搬 Hermes 的文件格式，而是复现它背后的系统分工：稳定层小而

三、四层记忆怎么抽取：来源、压缩和输出

有了四层理论之后，再回到素材。

`materials/simulated_long_conversation.jsonl` 是一段仿真长对话：三个会话、36 个事件，就是第一节小李那个 Agent 的完整流水。

会话	内容	埋了什么
<code>s_0614_beijing</code> (18 轮)	北京三日亲子游	路线检查失败→修正、用户偏好（午睡、别塞满）、显式交代"以后附检查结论"、天气闲聊
<code>s_0621_xian</code> (14 轮)	西安两日游	同一类跨区失败第二次出现、"重点排上午"偏好、口误噪声
<code>s_0622_expense</code> (4 轮)	出租车发票报销	另一个项目的显式交代"以后小额票直接预审"，用来验证记忆的项目隔离

每个事件带四类字段：身份（`event_id` / `project_id` / `session_id` / `turn`）、角色（`user` / `assistant` / `tool`）、内容（`text`），以及工具事件特有的结构化结果（`tool` / `status`）。留意 `status` 这个字段：路线检查的"failed / passed"是工具返回的结构化事实，后面它会成为代码可以直接核对的证据。

```
# 读取素材，并直接把三组对话展示出来
import asyncio
import json
import os
import re
import shutil
from collections import defaultdict
from collections.abc import Awaitable, Callable
from pathlib import Path
from typing import Any, TypeVar

from agently import Agently, TriggerFlow, TriggerFlowRuntimeData

T = TypeVar("T")

def locate_lesson_root() -> Path:
    candidates = [Path.cwd(), Path.cwd() / "harness_class" / "lessons" /
```

```

"Memory"]
    for candidate in candidates:
        if (candidate / "materials" /
"simulated_long_conversation.jsonl").is_file():
            return candidate
    raise RuntimeError("请从课程目录或项目根目录运行这个 notebook。")

LESSON_ROOT = locate_lesson_root()
MATERIAL_PATH = LESSON_ROOT / "materials" /
"simulated_long_conversation.jsonl"
DEMO_ROOT = LESSON_ROOT / ".demo_runs" / "notebook_inline"
MEMORY_DIR = DEMO_ROOT / "memory"

def show(data: Any) -> None:
    print(json.dumps(data, ensure_ascii=False, indent=2, default=str))

def read_jsonl(path: Path) -> list[dict[str, Any]]:
    return [json.loads(line) for line in path.read_text(encoding="utf-
8").splitlines() if line.strip()]

def write_jsonl(path: Path, rows: list[dict[str, Any]]) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text("\n".join(json.dumps(row, ensure_ascii=False) for row in
rows) + "\n", encoding="utf-8")

events = read_jsonl(MATERIAL_PATH)
sessions: dict[str, list[dict[str, Any]]] = defaultdict(list)
for event in events:
    sessions[event["session_id"]].append(event)

SESSION_TITLES = {
    "s_0614_beijing": "第一组: 北京三日亲子游",
    "s_0621_xian": "第二组: 西安两日游",
    "s_0622_expense": "第三组: 出租车发票报销",
}

def render_session(session_id: str) -> None:
    print(f"\n=== {SESSION_TITLES[session_id]} ===")
    for event in sorted(sessions[session_id], key=lambda item:
item["turn"]):
        tool = f" <{event['tool']}:{event['status']}>" if event.get("tool")
else ""
        print(f"{event['turn']:02d}. {event['role']}{tool}:
{event['text']}")

for session_id in SESSION_TITLES:
    render_session(session_id)

```

=== 第一组：北京三日亲子游 ===

01. user: 帮我规划一个北京三日亲子游，6月20号出发，带着5岁的女儿。最后要给我一份能真正走得通的行程。
02. assistant: 好的。先确认两件事：你们住在哪个区域？有没有一定要去的景点？
03. user: 住王府井附近。故宫肯定要去，孩子一直念叨环球影城。对了，北京最近热不热？
04. assistant: 六月下旬白天气温30度上下，早晚舒服一些，记得给孩子备防晒和水壶。景点方面我来排一版初稿。
05. user: 孩子才5岁，走不了太多路，下午两点到四点最好能回酒店午睡。
06. assistant: 记下了。初版行程：第一天上午天安门广场，下午国家博物馆；第二天上午故宫，下午环球影城；第三天上午景山公园，下午王府井步行街。
07. tool <route_checker:failed>: 路线检查失败：第二天下午从故宫（东城区）到环球影城（通州区）单程通勤约1.5小时，加上入园排队，对5岁儿童的下午时段不可执行。
08. assistant: 检查没通过，问题出在把跨区的大项目塞进了半天。调整思路：同区域景点聚合到同一天，环球影城这种远距离大项单独占一整天。
09. user: 对，另外我不想要那种网红打卡式的安排，一天别塞太满。
10. assistant: 修改后行程 v2：第一天 天安门-故宫-景山，同区域步行可达，下午两点回酒店午睡；第二天 环球影城一整天；第三天上午自然博物馆，下午机动。
11. tool <route_checker:passed>: 修正后路线检查通过：三天行程内跨区移动仅一次，每日步行量与午休窗口符合学龄前儿童条件。
12. user: 这版可以。以后给我这种结果的时候，把检查结论一起附上，我不想只看一段漂亮介绍。
13. assistant: 好的，以后输出行程会附上路线检查的结论和依据。
14. user: 顺便问一下，故宫门票是不是要提前预约？
15. assistant: 是的，故宫需要提前在官方渠道实名预约，暑期旺季建议提前七天蹲点，当天基本没票。
16. user: 好，第一天上午就定故宫，票我今晚约。
17. assistant: 已调整：第一天上午故宫、下午景山，午休窗口保留。最终版行程和检查结论已经打包给你。
18. user: 今天先到这，谢谢。

=== 第二组：西安两日游 ===

01. user: 上次北京的行程走得很顺利。这周末想带娃去西安两天，还是那个原则，别太累。
02. assistant: 好的。初版：第一天上午兵马俑，下午回城逛回民街和钟楼；第二天上午陕西历史博物馆，下午大雁塔。
03. tool <route_checker:failed>: 路线检查失败：兵马俑位于临潼区，距市区约40公里，往返加游览至少6小时；下午回城遇高峰，第一天行程超时，不可执行。
04. assistant: 又是同一类问题：远距离大项被排进了半天。调整：兵马俑单独占一整天，市区景点聚合到第二天。
05. user: 对了，娃上午精神最好，重要的景点尽量排上午。
06. assistant: 收到。v2：第一天兵马俑一整天，上午看核心俑坑；第二天上午陕西历史博物馆，下午大雁塔北广场喷泉，晚饭回民街。
07. tool <route_checker:passed>: 修正后路线检查通过：两天行程跨区移动一次，每日行程量符合学龄前儿童条件，午休窗口已预留。
08. user: 陕西历博的票好约吗？
09. assistant: 陕西历史博物馆免费不免票，旺季名额紧张，建议提前五天在官方公众号预约。
10. user: 行。对了你们这个报销流程.....哦不对，说错了，那是工作群的事。
11. assistant: 没关系。西安这边还有什么要调整的吗？
12. user: 这次也记得把检查结论附上。
13. assistant: 附上了：route_checker 通过，两天行程跨区一次，午休窗口已预留，重点景点都在上午。
14. user: 完美，就这么定。

=== 第三组：出租车发票报销 ===

01. user：帮我看看这张出租车发票能不能走差旅报销，金额86块。

02. assistant：可以先确认两点：是否在出差期间产生、发票抬头是否为公司全称。这两条满足就能提交。

03. user：是出差期间的，抬头也对。以后这种小额交通票，你直接按新版差旅制度帮我预审，别每次都问。

04. assistant：好的，以后小额交通票默认按新版差旅制度先做预审，再给你确认结果。

四层记忆的抽取方式

四层记忆不是同一个时间点生成的。`raw / consolidated / semantic` 主要由整理管线生成和治理；`working memory` 是新任务进来时，按任务目标和项目范围动态构建的。

记忆层	来源	抽取方式	压缩方式	输出例子
Raw Episodic	对话事件、工具调用、工具结果	代码原样写入，不做语义判断	不压缩，只保留原始事件	<code>raw_events.jsonl</code>
Consolidated Episodic	同一会话内的 raw 事件	agent direct request 读取事件流，抽取 episode 摘要和候选记忆	多轮对话压成会话摘要、失败原因、候选经验	<code>consolidated_sessions.jsonl</code>
Semantic / Policy	多个 episode 的候选记忆	agent 判断同义候选，代码按证据决定晋升	候选句压成稳定规	<code>semantic_rules.jsonl</code>

记忆层	来源	抽取方式	压缩方式	输出例子
			则， 并保 留晋 升理 由和 证据 链接	
Working Memory	当前任 务 + 可 召回的 长期记 忆	读取 semantic 和 consolidated 文件	规 则、 经 验、 证据 按相 关性 打包 成任 务简 报	task_brief.json

这个表是后面所有代码的地图。心跳整理只是定时触发前三层的抽取和晋升；新任务唤起时，系统再把前三层里相关的内容压成 working memory。

先只写 raw 层，看"没有自进化整理"的召回长什么样

这个实验故意只做一半：把 36 个事件原样写进 raw_events.jsonl，不做任何整理，然后用最朴素的文件检索看新任务能拿回什么。

```
# 只保存 raw 事件，然后用朴素文本重叠做一次召回
if DEMO_ROOT.exists():
    shutil.rmtree(DEMO_ROOT)
MEMORY_DIR.mkdir(parents=True, exist_ok=True)

NEW_TASK = "给一家三口规划上海两天家庭游，最终路线要可执行。"
RAW_PATH = MEMORY_DIR / "raw_events.jsonl"
write_jsonl(RAW_PATH, events)
```

```

def score_raw_event(event: dict[str, Any], query: str) -> int:
    text = str(event.get("text", ""))
    query_tokens = set(re.findall(r"[一-隹]{2}|[a-zA-Z0-9]+", query))
    return sum(1 for token in query_tokens if token in text)

travel_events = [event for event in events if event["project_id"] ==
"travel-agent"]
ranked_raw_events = sorted(
    travel_events,
    key=lambda event: (score_raw_event(event, NEW_TASK), -
int(event["turn"])),
    reverse=True,
)

show(
    {
        "raw_file": str(RAW_PATH.relative_to(LESSON_ROOT)),
        "raw_event_count": len(events),
        "new_task": NEW_TASK,
        "raw_recall_items": [
            {
                "event_id": event["event_id"],
                "session_id": event["session_id"],
                "role": event["role"],
                "text": event["text"],
            }
            for event in ranked_raw_events[:5]
        ],
    }
)

```

```

{
    "raw_file": ".demo_runs/notebook_inline/memory/raw_events.jsonl",
    "raw_event_count": 36,
    "new_task": "给一家三口规划上海两天家庭游，最终路线要可执行。",
    "raw_recall_items": [
        {
            "event_id": "evt_b03",
            "session_id": "s_0621_xian",
            "role": "tool",
            "text": "路线检查失败：兵马俑位于临潼区，距市区约40公里，往返加游览至少6小时；下午回城遇高峰，第一天行程超时，不可执行。"
        },
        {
            "event_id": "evt_a07",
            "session_id": "s_0614_beijing",
            "role": "tool",
            "text": "路线检查失败：第二天下午从故宫（东城区）到环球影城（通州区）单程通勤约1.5小时，加上入园排队，对5岁儿童的下午时段不可执行。"
        },
    ],
}

```

```

{
  "event_id": "evt_b07",
  "session_id": "s_0621_xian",
  "role": "tool",
  "text": "修正后路线检查通过：两天行程跨区移动一次，每日行程量符合学龄前儿童条件，午休窗口已预留。"
},
{
  "event_id": "evt_a11",
  "session_id": "s_0614_beijing",
  "role": "tool",
  "text": "修正后路线检查通过：三天行程内跨区移动仅一次，每日步行量与午休窗口符合学龄前儿童条件。"
},
{
  "event_id": "evt_a13",
  "session_id": "s_0614_beijing",
  "role": "assistant",
  "text": "好的，以后输出行程会附上路线检查的结论和依据。"
}
]
}

```

召回拿到的全是流水片段：两条失败报错、两条修正后的检查通过、一句“以后会附上结论”的应答。看起来条条沾边，但没有一条能直接遵守——“远距离大项单独占一整天”这条最值钱的教训，分散在失败报错、修正方案、用户确认三四个事件里，任何逐条检索都拼不回完整的它。问题不在检索算法，在于 raw 层里根本不存在这条“整理后的记录”。

这就是“分层”要解决的具体问题：值得留下的信息需要先被**抽取和整理成独立的记录**，才谈得上被召回。

分步整理：每一层都有暂存产物

下面不把四层整理挤进一个大脚本，而是分成几段可以单独观察的代码：raw 只保存，consolidated 负责会话压缩，semantic 负责规则晋升，working memory 负责新任务临时打包。

课堂讲义里用变量缓存中间结果，方便逐段观察；[scripts/02_collect_raw.py](#) 到 [scripts/05_build_working_memory.py](#) 保留文件暂存版本，适合课后按步骤运行和调试。

模型接口只配置一次：后面的抽取、合并和综合案例都会复用

```
def configure_model() -> None:
    try:
        from dotenv import load_dotenv
    except ImportError:
        load_dotenv = None
    if load_dotenv is not None:
        for directory in (LESSON_ROOT, *LESSON_ROOT.parents):
            env_path = directory / ".env"
            if env_path.is_file():
                load_dotenv(env_path, override=False)
                break

    api_key = os.getenv("DEEPSEEK_API_KEY")
    if not api_key:
        raise RuntimeError("需要 DEEPSEEK_API_KEY (放在课程根目录 .env 或 shell 导出)。")
    Agently.set_settings(
        "OpenAICompatible",
        {
            "base_url": os.getenv("DEEPSEEK_BASE_URL",
"https://api.deepseek.com/v1"),
            "api_key": api_key,
            "model": os.getenv("DEEPSEEK_DEFAULT_MODEL", "deepseek-chat"),
        },
    )

def is_transient_model_error(error: Exception) -> bool:
    message = str(error)
    return any(
        marker in message
        for marker in (
            "503",
            "Service is too busy",
            "service_unavailable",
            "Expected response header Content-Type",
            "transient_model_parse_failed",
        )
    )

async def run_model_request_with_retry(call: Callable[[], Awaitable[T]], *,
attempts: int = 4) -> T:
    for attempt in range(1, attempts + 1):
        try:
            return await call()
        except Exception as error:
            if attempt >= attempts or not is_transient_model_error(error):
                raise
            await asyncio.sleep(2 * attempt)
    raise RuntimeError("unreachable retry state")

configure_model()
```

```
print("model configured")
```

```
model configured
```

```
# consolidated 层: 让模型把一个会话压缩成摘要和候选记忆
async def extract_session_memories(session_events: list[dict[str, Any]]) -> dict[str, Any]:
    payload = [
        {
            "event_id": event["event_id"],
            "role": event["role"],
            "text": event["text"],
            **({"tool": event["tool"], "status": event["status"]} if
event.get("tool") else {}),
        }
        for event in session_events
    ]

    async def request() -> dict[str, Any]:
        agent = Agently.create_agent()
        result = await (
            agent.info({"对话事件流": payload})
            .input("对这段对话事件流做记忆抽取。")
            .instruct(
                [
                    "episode_summary: 三句话以内, 概括这次会话做了什么、失败过什么、怎么修正的。",
                    "memory_items: 只抽取以后任务还会用得上的信息, 分为 user_preference、fact、lesson 三类。",
                    "寒暄、闲聊、口误、只对当次有效的问答不要抽。",
                    "statement 写成脱离本次对话也能读懂的一句话。",
                    "supported_event_ids 只能取给定事件里的 event_id。",
                    "durable: 用户明确表示这条要求以后一直有效才是 true。",
                ]
            )
            .output(
                {
                    "episode_summary": ("str",),
                    "memory_items": [
                        {
                            "type": ("str", "user_preference | fact | lesson"),
                            "statement": ("str",),
                            "supported_event_ids": [("str",)],
                            "durable": ("bool", "用户是否显式要求长期生效"),
                        }
                    ],
                }
            )
        ).async_start()

    return result()
```

```

        if not isinstance(result, dict) or "episode_summary" not in result:
            raise RuntimeError("transient_model_parse_failed: memory
extraction returned invalid structure")
        return result

    return await run_model_request_with_retry(request)

```

```

# 执行 raw -> consolidated, 并把中间结果同时留在变量和文件里
raw_events = read_jsonl(RAW_PATH)
event_index_rows = [
    {
        "event_id": event["event_id"],
        "project_id": event["project_id"],
        "session_id": event["session_id"],
        "turn": event["turn"],
        "role": event["role"],
        "tool": event.get("tool"),
        "status": event.get("status"),
        "text": event["text"],
    }
    for event in raw_events
]

grouped_events: dict[tuple[str, str], list[dict[str, Any]]] =
defaultdict(list)
for event in raw_events:
    grouped_events[(event["project_id"], event["session_id"])].append(event)

consolidated_rows: list[dict[str, Any]] = []
candidate_rows: list[dict[str, Any]] = []
for (project_id, session_id), session_events in grouped_events.items():
    session_events.sort(key=lambda event: event["turn"])
    extraction = await extract_session_memories(session_events)
    memory_items = list(extraction.get("memory_items") or [])
    consolidated_rows.append(
        {
            "project_id": project_id,
            "session_id": session_id,
            "summary": extraction["episode_summary"],
            "source_event_ids": [event["event_id"] for event in
session_events],
            "candidate_count": len(memory_items),
        }
    )
    for index, item in enumerate(memory_items):
        candidate_rows.append(
            {
                "candidate_id": f"{session_id}#{index}",
                "project_id": project_id,
                "session_id": session_id,
                "type": str(item.get("type", "fact")),
                "statement": str(item.get("statement", "")),
                "durable": bool(item.get("durable")),
                "supported_event_ids": [event_id for event_id in

```

```

item.get("supported_event_ids", [])),
    }
)

write_jsonl(MEMORY_DIR / "consolidated_sessions.jsonl", consolidated_rows)
write_jsonl(MEMORY_DIR / "candidate_memories.jsonl", candidate_rows)
write_jsonl(MEMORY_DIR / "event_index.jsonl", event_index_rows)

show(
    {
        "session_count": len(consolidated_rows),
        "candidate_count": len(candidate_rows),
        "sample_consolidated": consolidated_rows[:2],
        "sample_candidates": candidate_rows[:5],
    }
)

```

```

{
    "session_count": 3,
    "candidate_count": 16,
    "sample_consolidated": [
        {
            "project_id": "travel-agent",
            "session_id": "s_0614_beijing",
            "summary": "用户规划了北京三日亲子游，初始方案因第二天下午跨区安排环球影城被路线检查工具判定为不可行，修正后将同区域景点集中、环球影城独占一整天并通过检查，最终采纳了第一天故宫和景山、第二天环球影城、第三天自然博物馆的行程。",
            "source_event_ids": [
                "evt_a01",
                "evt_a02",
                "evt_a03",
                "evt_a04",
                "evt_a05",
                "evt_a06",
                "evt_a07",
                "evt_a08",
                "evt_a09",
                "evt_a10",
                "evt_a11",
                "evt_a12",
                "evt_a13",
                "evt_a14",
                "evt_a15",
                "evt_a16",
                "evt_a17",
                "evt_a18"
            ],
            "candidate_count": 8
        },
        {

```

```
    "project_id": "travel-agent",
    "session_id": "s_0621_xian",
    "summary": "用户要求为周末西安两日游规划行程，初始方案因兵马俑距离市区远导致行程超
时被拒绝；修正后将兵马俑单独安排一整天、市区景点集中第二天，并遵从孩子上午精力好的偏好，最终
方案检查通过。",
    "source_event_ids": [
        "evt_b01",
        "evt_b02",
        "evt_b03",
        "evt_b04",
        "evt_b05",
        "evt_b06",
        "evt_b07",
        "evt_b08",
        "evt_b09",
        "evt_b10",
        "evt_b11",
        "evt_b12",
        "evt_b13",
        "evt_b14"
    ],
    "candidate_count": 5
}
],
"sample_candidates": [
    {
        "candidate_id": "s_0614_beijing#0",
        "project_id": "travel-agent",
        "session_id": "s_0614_beijing",
        "type": "user_preference",
        "statement": "5岁女儿下午两点到四点需要回酒店午睡，行程不能太满。",
        "durable": true,
        "supported_event_ids": [
            "evt_a05",
            "evt_a06"
        ]
    },
    {
        "candidate_id": "s_0614_beijing#1",
        "project_id": "travel-agent",
        "session_id": "s_0614_beijing",
        "type": "user_preference",
        "statement": "不喜欢网红打卡式安排，一天别塞太满。",
        "durable": true,
        "supported_event_ids": [
            "evt_a09"
        ]
    },
    {
        "candidate_id": "s_0614_beijing#2",
        "project_id": "travel-agent",
        "session_id": "s_0614_beijing",
```

```

    "type": "user_preference",
    "statement": "以后输出行程时要附上路线检查结论和依据。",
    "durable": true,
    "supported_event_ids": [
      "evt_a12",
      "evt_a13"
    ]
  },
  {
    "candidate_id": "s_0614_beijing#3",
    "project_id": "travel-agent",
    "session_id": "s_0614_beijing",
    "type": "fact",
    "statement": "用户住在王府井附近，计划6月20日出发。",
    "durable": false,
    "supported_event_ids": [
      "evt_a01",
      "evt_a03"
    ]
  },
  {
    "candidate_id": "s_0614_beijing#4",
    "project_id": "travel-agent",
    "session_id": "s_0614_beijing",
    "type": "fact",
    "statement": "用户想去的景点包括故宫、环球影城、景山公园、国家博物馆、自然博物馆
(最终版含自然博物馆)。",
    "durable": false,
    "supported_event_ids": [
      "evt_a03",
      "evt_a10"
    ]
  }
]
}

```

抽取脚本给每条候选记忆定了两个字段约束，值得停一下。第一，**statement**（记忆陈述）要求“脱离本次对话也能读懂”——记忆是给未来的任务读的，“用户说下午要午睡”到了下个月就没人知道指的是谁、哪次行程。第二，**supported_event_ids** 强制每条候选挂上支撑它的事件——没有证据的记忆后面既不能核对，也不能晋升。跑完后打开 [.demo_runs/notebook_inline/memory/candidate_memories.jsonl](#) 可以验证：天气闲聊、口误都没被抽进来。

语义层输出：同义合并与三条晋升通道

`candidate_memories.jsonl` 还不是长期规则。下一步先让模型合并同义候选，再由代码按证据决定是否晋升：跨会话重复、工具失败证据、用户显式长期指令，满足任一条才进入 `semantic_rules.jsonl`。

```
# semantic 层：模型合并同义候选，代码按证据决定是否晋升
SEMANTIC_KIND_BY_TYPE = {
    "user_preference": "user_preference_rule",
    "lesson": "project_rule",
    "fact": "stable_fact",
}

async def merge_candidates(candidates: list[dict[str, Any]]) ->
list[dict[str, Any]]:
    async def request() -> dict[str, Any]:
        agent = Agently.create_agent()
        result = await (
            agent.info(
                {
                    "候选记忆": [
                        {
                            "candidate_id": candidate["candidate_id"],
                            "type": candidate["type"],
                            "statement": candidate["statement"],
                        }
                        for candidate in candidates
                    ]
                }
            )
            .input("判断候选记忆里哪些说的是同一件事，把同义的合并成一条。")
            .instruct(
                [
                    "同一个偏好、同一条做法、同一个事实的不同表述要合并；",
                    "合并后的 statement 是一条规则，不是原句拼接；",
                    "member_ids 列出被合并进来的 candidate_id；",
                    "独立的候选也要输出，member_ids 只放它自己；",
                    "不要发明候选里没有的内容。",
                ]
            )
            .output({"merged_memories": [{"statement": ("str",), "type":
("str",), "member_ids": [("str",)]}]}))
            .async_start()
        )
        if not isinstance(result, dict) or not
isinstance(result.get("merged_memories"), list):
            raise RuntimeError("transient_model_parse_failed: merge returned
invalid structure")
        return result

    result = await run_model_request_with_retry(request)
    return result["merged_memories"]

def promotion_reason(merged_type: str, members: list[dict[str, Any]],
```

```

supporting_event_ids: list[str]) -> str | None:
    supporting_sessions = {member["session_id"] for member in members}
    event_by_id = {event["event_id"]: event for event in event_index_rows}
    failure_backed = any(event_by_id.get(event_id, {}).get("status") ==
"failed" for event_id in supporting_event_ids)
    explicit_durable = merged_type == "user_preference" and
any(member.get("durable") for member in members)
    if len(supporting_sessions) >= 2:
        return "repeated_across_sessions"
    if failure_backed:
        return "tool_failure_evidence"
    if explicit_durable:
        return "explicit_user_instruction"
    return None

```

```

# 执行 candidate -> semantic / kept
semantic_rows: list[dict[str, Any]] = []
kept_rows: list[dict[str, Any]] = []
candidates_by_project: dict[str, list[dict[str, Any]]] = defaultdict(list)
for candidate in candidate_rows:
    candidates_by_project[candidate["project_id"]].append(candidate)

episode_by_session = {row["session_id"]: row for row in consolidated_rows}
rule_index = 1
for project_id, project_candidates in candidates_by_project.items():
    merged_memories = await merge_candidates(project_candidates)
    candidate_by_id = {candidate["candidate_id"]: candidate for candidate in
project_candidates}
    for merged in merged_memories:
        members = [candidate_by_id[mid] for mid in merged.get("member_ids",
[])] if mid in candidate_by_id
        if not members:
            continue
        supporting_event_ids = sorted({eid for member in members for eid in
member["supported_event_ids"]})
        supporting_sessions = sorted({member["session_id"] for member in
members})
        reason = promotion_reason(str(merged.get("type")), members,
supporting_event_ids)
        if reason is None:
            kept_rows.append({"project_id": project_id, "statement":
merged["statement"], "reason": "not_enough_evidence"})
            continue
        semantic_rows.append(
            {
                "rule_id": f"rule_{rule_index:03d}",
                "project_id": project_id,
                "kind": SEMANTIC_KIND_BY_TYPE.get(str(merged.get("type")),
"project_rule"),
                "rule": merged["statement"],
                "promotion_reason": reason,
                "supporting_event_ids": supporting_event_ids,
                "supporting_sessions": supporting_sessions,
                "derived_from": [

```

```

        {"session_id": sid, "summary":
episode_by_session.get(sid, {}).get("summary", "")}
        for sid in supporting_sessions
    ],
    }
)
rule_index += 1

promotion_report = {
    "semantic_count": len(semantic_rows),
    "kept_count": len(kept_rows),
    "rules": [{"rule_id": row["rule_id"], "reason": row["promotion_reason"],
"rule": row["rule"]} for row in semantic_rows],
}
write_jsonl(MEMORY_DIR / "semantic_rules.jsonl", semantic_rows)
write_jsonl(MEMORY_DIR / "kept_candidates.jsonl", kept_rows)
(MEMORY_DIR /
"promotion_report.json").write_text(json.dumps(promotion_report,
ensure_ascii=False, indent=2), encoding="utf-8")
show(promotion_report)

```

```

{
  "semantic_count": 8,
  "kept_count": 6,
  "rules": [
    {
      "rule_id": "rule_001",
      "reason": "explicit_user_instruction",
      "rule": "5岁女儿下午两点到四点需要回酒店午睡，行程不能太满。"
    },
    {
      "rule_id": "rule_002",
      "reason": "explicit_user_instruction",
      "rule": "不喜欢网红打卡式安排，一天不要塞太满。"
    },
    {
      "rule_id": "rule_003",
      "reason": "explicit_user_instruction",
      "rule": "输出行程时要附上路线检查结论和依据。"
    },
    {
      "rule_id": "rule_004",
      "reason": "tool_failure_evidence",
      "rule": "当天下午从故宫（东城区）跨区到环球影城（通州区）单程通勤约1.5小时，加上排队对5岁儿童不可行。"
    },
    {
      "rule_id": "rule_005",
      "reason": "explicit_user_instruction",
      "rule": "希望旅行行程不要太累，适合带孩子。"
    },
  ],
}

```

```

{
  "rule_id": "rule_006",
  "reason": "explicit_user_instruction",
  "rule": "重要景点尽量安排在上午，因为孩子上午精神最好。",
},
{
  "rule_id": "rule_007",
  "reason": "tool_failure_evidence",
  "rule": "兵马俑位于临潼区，距市区约40公里，往返加游览至少6小时，不能与下午的市区景点排在同一天。",
},
{
  "rule_id": "rule_008",
  "reason": "explicit_user_instruction",
  "rule": "每次行程规划后，需要附上路线检查结论。",
}
]
}

```

working memory: 按当前任务挑选稳定规则和相关 episode

TASK_BRIEF_PATH = MEMORY_DIR / "task_brief.json"

PROJECT_ID = "travel-agent"

TASK_FOR_RECALL = "带5岁的女儿去上海玩两天，孩子一直想去迪士尼乐园，帮我出行程，最终路线要可执行。"

```
def score_text(text: str, query: str) -> int:
```

```
    keywords = ["孩子", "亲子", "路线", "检查", "远距离", "迪士尼", "环球", "兵马俑", "午休", "午睡"]
```

```
    return sum(2 for keyword in keywords if keyword in text and keyword in query) + sum(
        1 for keyword in keywords if keyword in text
    )
```

```
semantic_for_task = [row for row in semantic_rows if row["project_id"] == PROJECT_ID]
```

```
consolidated_for_task = [row for row in consolidated_rows if row["project_id"] == PROJECT_ID]
```

```
ranked_episodes = sorted(consolidated_for_task, key=lambda row: score_text(row["summary"], TASK_FOR_RECALL), reverse=True)
```

```
task_brief = {
```

```
    "task": TASK_FOR_RECALL,
```

```
    "project_id": PROJECT_ID,
```

```
    "stable_rules": [
```

```
        {
```

```
            "rule_id": row["rule_id"],
```

```
            "kind": row["kind"],
```

```
            "rule": row["rule"],
```

```
            "promotion_reason": row["promotion_reason"],
```

```
        }
```

```
    for row in semantic_for_task
```

```

],
"selected_episodes": [
    {"session_id": row["session_id"], "summary": row["summary"]}
    for row in ranked_episodes[:2]
],
"source_files": [
    str((MEMORY_DIR / "semantic_rules.jsonl").relative_to(LESSON_ROOT)),
    str((MEMORY_DIR /
"consolidated_sessions.jsonl").relative_to(LESSON_ROOT)),
],
}
TASK_BRIEF_PATH.write_text(json.dumps(task_brief, ensure_ascii=False,
indent=2), encoding="utf-8")
show(task_brief)

```

```

{
  "task": "带5岁的女儿去上海玩两天，孩子一直想去迪士尼乐园，帮我出行程，最终路线要可执行。",
  "project_id": "travel-agent",
  "stable_rules": [
    {
      "rule_id": "rule_001",
      "kind": "user_preference_rule",
      "rule": "5岁女儿下午两点到四点需要回酒店午睡，行程不能太满。",
      "promotion_reason": "explicit_user_instruction"
    },
    {
      "rule_id": "rule_002",
      "kind": "user_preference_rule",
      "rule": "不喜欢网红打卡式安排，一天不要塞太满。",
      "promotion_reason": "explicit_user_instruction"
    },
    {
      "rule_id": "rule_003",
      "kind": "user_preference_rule",
      "rule": "输出行程时要附上路线检查结论和依据。",
      "promotion_reason": "explicit_user_instruction"
    },
    {
      "rule_id": "rule_004",
      "kind": "project_rule",
      "rule": "当天下午从故宫（东城区）跨区到环球影城（通州区）单程通勤约1.5小时，加上排队对5岁儿童不可行。",
      "promotion_reason": "tool_failure_evidence"
    },
    {
      "rule_id": "rule_005",
      "kind": "user_preference_rule",
      "rule": "希望旅行行程不要太累，适合带孩子。",
      "promotion_reason": "explicit_user_instruction"
    }
  ]
}

```

```

    },
    {
      "rule_id": "rule_006",
      "kind": "user_preference_rule",
      "rule": "重要景点尽量安排在上午，因为孩子上午精神最好。",
      "promotion_reason": "explicit_user_instruction"
    },
    {
      "rule_id": "rule_007",
      "kind": "project_rule",
      "rule": "兵马俑位于临潼区，距市区约40公里，往返加游览至少6小时，不能与下午的市区景点排在同一天。",
      "promotion_reason": "tool_failure_evidence"
    },
    {
      "rule_id": "rule_008",
      "kind": "user_preference_rule",
      "rule": "每次行程规划后，需要附上路线检查结论。",
      "promotion_reason": "explicit_user_instruction"
    }
  ],
  "selected_episodes": [
    {
      "session_id": "s_0614_beijing",
      "summary": "用户规划了北京三日亲子游，初始方案因第二天下午跨区安排环球影城被路线检查工具判定为不可行，修正后将同区域景点集中、环球影城独占一整天并通过检查，最终采纳了第一天故宫和景山、第二天环球影城、第三天自然博物馆的行程。"
    },
    {
      "session_id": "s_0621_xian",
      "summary": "用户要求为周末西安两日游规划行程，初始方案因兵马俑距离市区远导致行程超时被拒绝；修正后将兵马俑单独安排一整天、市区景点集中第二天，并遵从孩子上午精力好的偏好，最终方案检查通过。"
    }
  ],
  "source_files": [
    ".demo_runs/notebook_inline/memory/semantic_rules.jsonl",
    ".demo_runs/notebook_inline/memory/consolidated_sessions.jsonl"
  ]
}

```

四、心跳整理：定时触发第三节的抽取管线

到这里，四层记忆的来源、压缩和输出已经讲清楚了。心跳机制做的事就简单很多：它不是新的记忆理论，而是一个后台定时任务，负责在系统空闲时扫描新 raw 事件，然后触发第三节那套抽取、合并、晋升和 checkpoint 流程。

整理不适合放在对话进行中做——用户还在等回复，没人愿意每轮多花几秒钟等 Agent"记笔记"。更合理的位置是后台：对话结束后、系统空闲时，由一个定时唤醒的整理器扫描新事件、抽取值得留下的信息。本文沿用 OpenClaw 一类个人 Agent 里的叫法，把这个后台触发器称为 heartbeat（心跳）。

管线：一次心跳做四件事

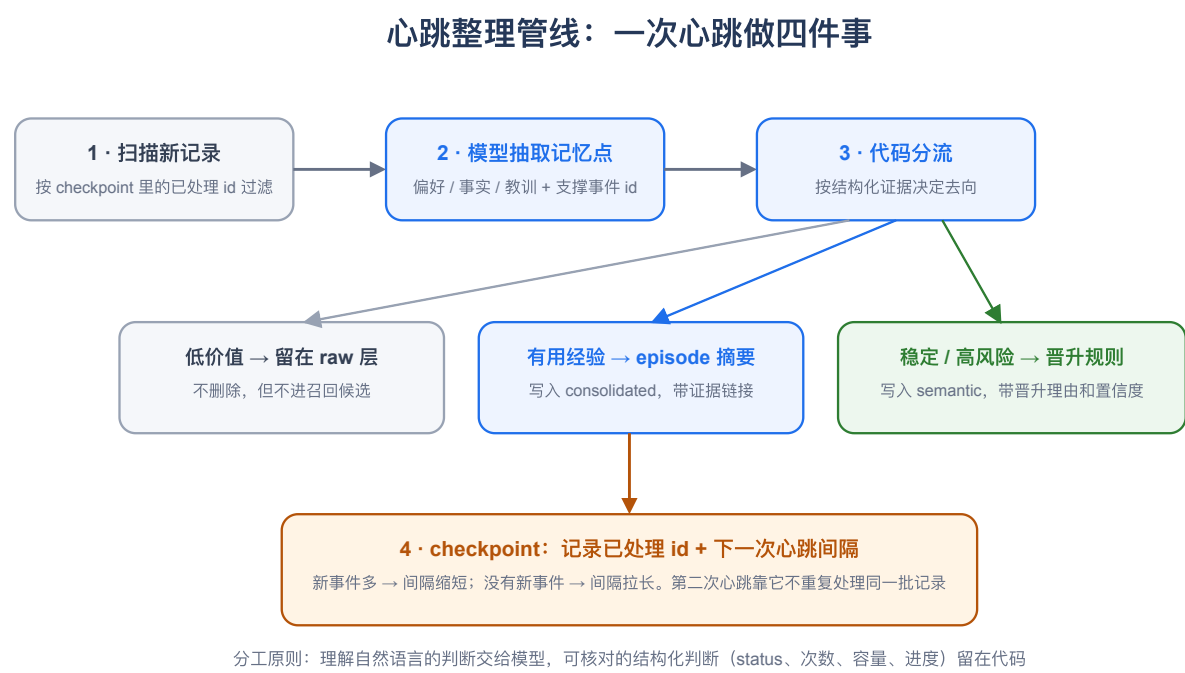


图 3：扫描新记录 → 模型抽取记忆点 → 代码分流 → checkpoint 记录进度和下次间隔。

分工原则在这条管线里落得很具体。**模型负责的判断**：这段对话里哪几句值得长期记住？两条措辞不同的候选是不是同一件事？用户是不是显式说了"以后都这样"？这些都是理解自然语言，代码里写 `if "失败" in text` 这类关键词匹配是靠不住的——换个说法就漏。**代码负责的判断**：工具返回的 `status` 是不是 failed？一条候选被几个会话独立支持？处理到哪条了？这些是可核对的结构化事实，交给模型反而引入不确定性。

心跳本体：扫描 checkpoint，触发文件整理步骤

有了前面的整理函数之后，心跳本体就简单了：它不负责抽取逻辑，只负责醒来、检查有没有新 raw 事件、有新事件时触发同一套整理函数、最后写回 `heartbeat_state.json`。课后脚本里的对应入口是 `scripts/06_heartbeat_trigger.py`。

```
# 心跳：用 TriggerFlow 扫描新 raw 事件，再触发同一套整理结果写回 checkpoint
HEARTBEAT_STATE_PATH = DEMO_ROOT / "heartbeat_state.json"
```

```

def read_heartbeat_state() -> dict[str, Any]:
    if not HEARTBEAT_STATE_PATH.is_file():
        return {"processed_event_ids": []}
    return json.loads(HEARTBEAT_STATE_PATH.read_text(encoding="utf-8"))

def write_heartbeat_state(state: dict[str, Any]) -> None:
    HEARTBEAT_STATE_PATH.write_text(json.dumps(state, ensure_ascii=False,
indent=2), encoding="utf-8")

def adaptive_interval_seconds(new_count: int) -> int:
    if new_count >= 10:
        return 60
    if new_count >= 1:
        return 300
    return 900

async def scan_raw_events(data: TriggerFlowRuntimeData) -> dict[str, Any]:
    raw_event_ids = [event["event_id"] for event in read_jsonl(RAW_PATH)]
    data.set_state("raw_event_ids", raw_event_ids, emit=False)
    return {"raw_event_count": len(raw_event_ids)}

async def compare_checkpoint(data: TriggerFlowRuntimeData) -> dict[str,
Any]:
    state = read_heartbeat_state()
    processed = set(state.get("processed_event_ids", []))
    new_event_ids = [event_id for event_id in
data.get_state("raw_event_ids", []) if event_id not in processed]
    data.set_state("previous_state", state, emit=False)
    data.set_state("new_event_ids", new_event_ids, emit=False)
    return {"new_event_count": len(new_event_ids)}

async def trigger_existing_pipeline(data: TriggerFlowRuntimeData) ->
dict[str, Any]:
    new_event_ids = list(data.get_state("new_event_ids", []) or [])
    steps = ["raw->consolidated", "candidate->semantic", "semantic+episode-
>working_memory"]
    data.set_state("steps_ran", steps if new_event_ids else [], emit=False)
    return {"skipped": not bool(new_event_ids), "steps_ran": steps if
new_event_ids else []}

async def write_heartbeat_checkpoint(data: TriggerFlowRuntimeData) ->
dict[str, Any]:
    raw_event_ids = list(data.get_state("raw_event_ids", []) or [])
    new_event_ids = list(data.get_state("new_event_ids", []) or [])
    previous_state = dict(data.get_state("previous_state", {}) or {})
    processed = sorted(set(previous_state.get("processed_event_ids", [])) |
set(new_event_ids))
    report = {
        "new_raw_count": len(new_event_ids),
        "processed_event_count": len(processed),
    }

```

```

        "steps_ran": list(data.get_state("steps_ran", []) or []),
        "next_interval_seconds":
adaptive_interval_seconds(len(new_event_ids)),
    }
    write_heartbeat_state({"processed_event_ids": processed or
raw_event_ids, "last_report": report})
    data.set_state("heartbeat_report", report, emit=False)
    return report

flow = TriggerFlow(name="notebook-file-memory-heartbeat")
flow.to(scan_raw_events, name="scan_raw_events").to(compare_checkpoint,
name="compare_checkpoint").to(
    trigger_existing_pipeline, name="trigger_existing_pipeline"
).to(write_heartbeat_checkpoint, name="write_heartbeat_checkpoint")

async def heartbeat_once() -> dict[str, Any]:
    execution = flow.create_execution(workspace=False, concurrency=1)
    await execution.async_start({"trigger": "heartbeat"})
    state = await execution.async_close()
    report = state.get("heartbeat_report")
    if not isinstance(report, dict):
        raise RuntimeError("heartbeat did not produce a report")
    return report

if HEARTBEAT_STATE_PATH.exists():
    HEARTBEAT_STATE_PATH.unlink()
first_heartbeat = await heartbeat_once()
second_heartbeat = await heartbeat_once()
show({"first_heartbeat": first_heartbeat, "second_heartbeat":
second_heartbeat})

```

```

{
  "first_heartbeat": {
    "new_raw_count": 36,
    "processed_event_count": 36,
    "steps_ran": [
      "raw->consolidated",
      "candidate->semantic",
      "semantic+episode->working_memory"
    ],
    "next_interval_seconds": 60
  },
  "second_heartbeat": {
    "new_raw_count": 0,
    "processed_event_count": 36,
    "steps_ran": [],
    "next_interval_seconds": 900
  }
}

```

```
show(json.loads(HEARTBEAT_STATE_PATH.read_text(encoding="utf-8")))
```

```
{
  "processed_event_ids": [
    "evt_a01",
    "evt_a02",
    "evt_a03",
    "evt_a04",
    "evt_a05",
    "evt_a06",
    "evt_a07",
    "evt_a08",
    "evt_a09",
    "evt_a10",
    "evt_a11",
    "evt_a12",
    "evt_a13",
    "evt_a14",
    "evt_a15",
    "evt_a16",
    "evt_a17",
    "evt_a18",
    "evt_b01",
    "evt_b02",
    "evt_b03",
    "evt_b04",
    "evt_b05",
    "evt_b06",
    "evt_b07",
    "evt_b08",
    "evt_b09",
    "evt_b10",
    "evt_b11",
    "evt_b12",
    "evt_b13",
    "evt_b14",
    "evt_c01",
    "evt_c02",
    "evt_c03",
    "evt_c04"
  ],
  "last_report": {
    "new_raw_count": 0,
    "processed_event_count": 36,
    "steps_ran": [],
    "next_interval_seconds": 900
  }
}
```

```
}  
}
```

值得注意的是没晋升的那批："住王府井附近"这类信息只出现过一次、没有失败证据、用户也没说"以后都这样"——它留在 episode 里等下次证据，而不是急着变成规则。晋升门槛宁紧勿松：**semantic 层进 prompt 的优先级最高**，进去一条错的，以后每个任务都要为它买单。

模型输出每次运行会有措辞差异，但结构应该稳定：跨区失败的教训走 repeated_across_sessions 或 tool_failure_evidence 通道，"附检查结论""小额票直接预审"这类显式交代走 explicit_user_instruction 通道。

心跳的安全边界：自动整理不能变成自动污染

心跳是后台进程，它读到的内容不只有用户对话——邮件、消息、网页、仓库都可能流进来。后台写记忆这条通道如果没有治理，会出安全问题，而且已经有测量数据：

证据 · Mind Your HEARTBEAT!：后台执行静默污染记忆的测量

来源 arXiv 2603.23064 (2026-03), *Mind Your HEARTBEAT! Claw Background Execution Inherently Enables Silent Memory Pollution*; 实验对象含 2026 年 2 月修复前的 OpenClaw

论文针对 Claw 系个人 Agent 的共同架构——heartbeat 后台执行和前台对话共用同一个会话与记忆通道——形式化了一条 E→M→B 路径：后台接触的不可信内容 (Exposure) 进入会话上下文、被例行的"随手存记忆"写成长期记忆 (Memory)，再在之后的前台任务里改变行为 (Behavior)。

测量结果：带社交可信度包装的误导内容，行为误导率最高 61%；例行记忆保存把短期污染固化进长期记忆的比例最高 91%；跨会话行为影响最高 76%。关键的一点：**不需要提示注入，普通的社交谣言就够了**——问题出在架构，不在某条恶意 prompt。

含义：后台写入必须有独立治理：来源标记、证据链接、置信度、可回滚。本文管线里 source / supported_event_ids / promotion_reason 这些字段，就是在给每条记忆留"它凭什么在这里"的答案。

注意边界：实验对象是个人助理形态的 Claw 生态（含修复前版本），数字不能直接外推到所有 Agent 架构；它证明的是"后台通道缺治理时污染是常态"，不是"心跳机

制不可用"。

心跳不是越主动越好。没有来源、没有证据、没有回滚路径的后台记忆写入，会把"自动整理"变成"自动污染"。

五、新任务唤起：从文件记忆构建 working memory

到这里，长期记忆已经被整理成几个可追溯的文件产物。新任务唤起要做的事，是读取 semantic 规则和 consolidated 摘要，压成当前任务可用的 `task_brief.json`，而不是把历史全部塞回上下文。

召回的四步



图 5：推导任务目标 → 分优先级取候选 → 打包为任务上下文 → 带着记忆执行。

文件版实现不需要复杂 API：先按 `project_id` 做项目隔离，再按任务目标挑选相关 episode，最后把稳定规则和候选经验打包成一个任务简报。重点在路由和边界，而不是某个存储后端。

文件产物：让进化后的记忆能被新任务找到

文件版的记忆库一共就这几个产物：

- `raw_events.jsonl` 保存完整事件流；
- `consolidated_sessions.jsonl` 保存会话摘要和候选数量；
- `candidate_memories.jsonl` 保存模型抽取出的候选记忆；
- `semantic_rules.jsonl` 保存被代码按证据晋升的稳定规则；
- `task_brief.json` 保存当前新任务要带入上下文的 working memory。

这些文件在 notebook 里写到 `.demo_runs/notebook_inline/memory/`；课后脚本版写到 `.demo_runs/file_memory_layers/memory/`。

```
# 新任务唤起：读取刚刚生成的 task_brief，而不是回灌全部历史
loaded_task_brief = json.loads(TASK_BRIEF_PATH.read_text(encoding="utf-8"))
show(
    {
        "task": loaded_task_brief["task"],
        "stable_rules": [rule["rule"] for rule in
loaded_task_brief["stable_rules"]],
        "selected_episodes": loaded_task_brief["selected_episodes"],
        "source_files": loaded_task_brief["source_files"],
        "observation": "新任务拿到的是整理后的规则和摘要，不再是 raw 对话流水。",
    }
)
```

```
{
    "task": "带5岁的女儿去上海玩两天，孩子一直想去迪士尼乐园，帮我出行程，最终路线要可执行。",
    "stable_rules": [
        "5岁女儿下午两点到四点需要回酒店午睡，行程不能太满。",
        "不喜欢网红打卡式安排，一天不要塞太满。",
        "输出行程时要附上路线检查结论和依据。",
        "当天下午从故宫（东城区）跨区到环球影城（通州区）单程通勤约1.5小时，加上排队对5岁儿童不可行。",
        "希望旅行行程不要太累，适合带孩子。",
        "重要景点尽量安排在上午，因为孩子上午精神最好。",
        "兵马俑位于临潼区，距市区约40公里，往返加游览至少6小时，不能与下午的市区景点排在同一天。",
        "每次行程规划后，需要附上路线检查结论。"
    ],
    "selected_episodes": [
        {
            "session_id": "s_0614_beijing",
            "summary": "用户规划了北京三日亲子游，初始方案因第二天下午跨区安排环球影城被路线检查工具判定为不可行，修正后将同区域景点集中、环球影城独占一整天并通过检查，最终采纳了第一天故宫和景山、第二天环球影城、第三天自然博物馆的行程。"
        },
        {
            "session_id": "s_0621_xian",
```

```

        "summary": "用户要求为周末西安两日游规划行程，初始方案因兵马俑距离市区远导致行程超
        时被拒绝；修正后将兵马俑单独安排一整天、市区景点集中第二天，并遵从孩子上午精力好的偏好，最终
        方案检查通过。"
    }
],
"source_files": [
    ".demo_runs/notebook_inline/memory/semantic_rules.jsonl",
    ".demo_runs/notebook_inline/memory/consolidated_sessions.jsonl"
],
"observation": "新任务拿到的是整理后的规则和摘要，不再是 raw 对话流水。"
}

```

文件版的召回已经能说明问题：稳定规则整批带入，episode 按项目和相关性挑选，raw 只在需要证据时回读。向量库解决的是“自然语言近义匹配”这一层增强，[extensions/chromadb_hybrid_retrieval.py](#) 给了原生 ChromaDB 示例；主事实仍然留在 JSON/JSONL 文件里。

六、综合案例：文件记忆接入 Workspace 后执行新任务

到目前为止，整套记忆都只是本地文件。综合案例把这些文件产物导入 Workspace：文件产物负责记忆治理，Workspace 负责持久化记录、scope 隔离和 ContextPackage 构建。

```

# 综合案例第一步：把变量里的文件记忆导入 Workspace
WORKSPACE_ROOT = DEMO_ROOT / "workspace"
if WORKSPACE_ROOT.exists():
    shutil.rmtree(WORKSPACE_ROOT)
workspace = Agently.create_workspace(WORKSPACE_ROOT)
SEMANTIC_COLLECTION = "memory-semantic"
CONSOLIDATED_COLLECTION = "memory-consolidated"

async def import_memory_to_workspace() -> None:
    for row in consolidated_rows:
        await workspace.ingest(
            content=row,
            collection=CONSOLIDATED_COLLECTION,
            kind="episode_summary",
            summary=row["summary"],
            scope={"project_id": row["project_id"], "session_id":
row["session_id"]},
            source={"type": "notebook_memory_pipeline", "file":
"consolidated_sessions.jsonl"},
            meta={"candidate_count": row.get("candidate_count", 0)},
        )
    for row in semantic_rows:

```

```

        await workspace.ingest(
            content=row,
            collection=SEMANTIC_COLLECTION,
            kind=row["kind"],
            summary=row["rule"],
            scope={"project_id": row["project_id"]},
            source={"type": "notebook_memory_pipeline", "file":
"semantic_rules.jsonl", "rule_id": row["rule_id"]},
            meta={"promotion_reason": row["promotion_reason"]},
        )

async def build_workspace_task_brief(task: str) -> dict[str, Any]:
    stable_rules = await workspace.search(None, filters={"collection":
SEMANTIC_COLLECTION, "scope.project_id": PROJECT_ID})
    context_pack = await workspace.build_context(
        goal=task,
        scope={"project_id": PROJECT_ID},
        budget={"chars": 1600, "item_chars": 400},
        profile="auto",
    )
    return {
        "task": task,
        "stable_rules": [record["summary"] for record in stable_rules],
        "selected_memory": [
            {
                "record_id": item["ref"]["id"],
                "collection": item["ref"]["collection"],
                "kind": item["kind"],
                "summary": item["summary"],
            }
            for item in context_pack["items"][:5]
        ],
        "candidate_count":
context_pack["diagnostics"].get("candidate_count"),
    }

await import_memory_to_workspace()
workspace_task_brief = await build_workspace_task_brief(TASK_FOR_RECALL)
show(workspace_task_brief)

```

```

{
    "task": "带5岁的女儿去上海玩两天，孩子一直想去迪士尼乐园，帮我出行程，最终路线要可执行。",
    "stable_rules": [
        "5岁女儿下午两点到四点需要回酒店午睡，行程不能太满。",
        "不喜欢网红打卡式安排，一天不要塞太满。",
        "输出行程时要附上路线检查结论和依据。",
        "当天下午从故宫（东城区）跨区到环球影城（通州区）单程通勤约1.5小时，加上排队对5岁儿童不可行。",
        "希望旅行行程不要太累，适合带孩子。",
        "重要景点尽量安排在上午，因为孩子上午精神最好。",
        "兵马俑位于临潼区，距市区约40公里，往返加游览至少6小时，不能与下午的市区景点排在同一

```

```
天。",
  "每次行程规划后，需要附上路线检查结论。",
],
"selected_memory": [
  {
    "record_id": "rec_6e271c8e18da4d1db793b682660c8a33",
    "collection": "memory-consolidated",
    "kind": "episode_summary",
    "summary": "用户规划了北京三日亲子游，初始方案因第二天下午跨区安排环球影城被路线检查工具判定为不可行，修正后将同区域景点集中、环球影城独占一整天并通过检查，最终采纳了第一天故宫和景山、第二天环球影城、第三天自然博物馆的行程。",
  },
  {
    "record_id": "rec_e68edd83c8614e619be65d40d47cdf6d",
    "collection": "memory-consolidated",
    "kind": "episode_summary",
    "summary": "用户要求为周末西安两日游规划行程，初始方案因兵马俑距离市区远导致行程超时被拒绝；修正后将兵马俑单独安排一整天、市区景点集中第二天，并遵从孩子上午精力好的偏好，最终方案检查通过。",
  },
  {
    "record_id": "rec_a9ba668d4030415fa62493239fe50335",
    "collection": "memory-semantic",
    "kind": "user_preference_rule",
    "summary": "5岁女儿下午两点到四点需要回酒店午睡，行程不能太满。",
  }
],
"candidate_count": 10
}
```

这里有两个观察点。第一，`expense-agent` 的小额报销规则不会被召回，因为 `Workspace` 的 `scope.project_id` 卡住了项目边界。第二，`Workspace` 不是记忆理论本身，而是把文件记忆接入工程系统的后端：文件产物负责记忆治理，`Workspace` 负责记录、隔离、召回和上下文打包。

上海亲子游的记忆增强对比

综合案例输出同一个任务的两版草案：不带记忆的 `draft_without_memory` 和带 `task_brief` 的 `draft_with_memory`。输出 `schema` 里专门留了一个“规则遵守”字段，两侧共用：不带记忆的版本没有规则可遵守，这个字段自然是空的；带记忆的版本要逐条交代 `stable_rules` 怎么落实。把“遵守情况”做成结构化字段而不是散在文字里，验收就从“人眼扫一遍”变成“逐条核对”。

观察点集中在三条规则有没有生效：

- 迪士尼（远距离大项）有没有**独占一整天**，而不是和市区景点混排；
- 有没有**留出午休窗口**、行程不塞满，重点安排放在上午；
- "规则遵守"字段里有没有****"附路线检查结论"**这一条的落实方式**。

有一点要提醒：模型输出每次运行有差异，个别规则靠通识也可能撞对——迪士尼独占一天就属于通识给得出的。稳定的差异来自通识给不出的那部分：不带记忆的版本不会主动留出午休窗口，更不会知道"这个用户要看检查结论"。这组对照就是整条管线的验收——行为差异来自记忆，而不是来自模型通识。

```
# 综合案例第二步：同一个任务分别在“不带记忆 / 带记忆”两种上下文下生成草案
async def draft_itinerary(task: str, memory_brief: dict[str, Any] | None) -> dict[str, Any]:
    stable_rules = list((memory_brief or {}).get("stable_rules", []))
    agent = Agently.create_agent()
    if stable_rules:
        agent.info({"必须遵守的长期规则": stable_rules, "可参考的过往经验":
memory_brief.get("selected_memory", [])})
        agent.instruct(
            [
                "长期规则来自记忆系统，不是本次用户输入里的普通要求；",
                "规则遵守字段必须逐条对应长期规则，格式为：规则原文 -> 本次怎么满足；",
                "如果规则提到孩子午睡，行程骨架必须写出 14:00-16:00 的休息窗口；",
                "如果规则提到远距离大项，迪士尼必须独占一整天；",
                "如果规则要求附路线检查结论，输出附带里要包含以'路线检查结论：'开头的说
明。",
            ]
        )
    else:
        agent.instruct("本次没有长期记忆简报。规则遵守字段只用于长期记忆规则，因此必须
返回空列表。")
    result = await (
        agent.input(task)
        .output(
            {
                "行程骨架": [("str", "每天一句话，说明主要安排")],
                "输出附带": [("str", "除行程本身外附带的提示")],
                "规则遵守": [("str", "长期记忆规则的落实情况；没有长期规则时为空")],
            }
        )
        .async_start()
    )
    if not isinstance(result, dict) or "行程骨架" not in result:
        raise RuntimeError("transient_model_parse_failed: itinerary draft
returned invalid structure")
    return result
```

```
draft_without_memory = await run_model_request_with_retry(lambda:
draft_itinerary(TASK_FOR_RECALL, None))
draft_with_memory = await run_model_request_with_retry(lambda:
draft_itinerary(TASK_FOR_RECALL, workspace_task_brief))
show(
```

```

{
    "workspace_root": str(WORKSPACE_ROOT.relative_to(LESSON_ROOT)),
    "draft_without_memory": draft_without_memory,
    "draft_with_memory": draft_with_memory,
}
)

```

```

{
    "workspace_root": ".demo_runs/notebook_inline/workspace",
    "draft_without_memory": {
        "行程骨架": [
            "第一天：上午从出发地抵达上海，入住迪士尼附近酒店，下午逛迪士尼小镇、体验儿童游乐区，晚上早睡养精蓄锐。",
            "第二天：全天游玩上海迪士尼乐园，按低龄儿童友好路线（小飞象→梦幻世界→小熊维尼）游玩，傍晚离园返程。"
        ],
        "输出附带": [
            "建议预订迪士尼乐园酒店或玩具总动员酒店，方便孩子午休和提前入园。",
            "给女儿带好防晒、水壶、轻便推车，乐园可租童车。",
            "提前下载上海迪士尼官方App，查看实时排队时间和演出时间。",
            "迪士尼小镇的儿童游乐区免费且适合低龄孩子，第一天可充分放电。",
            "第二天入园后先去梦幻世界，项目温和，孩子喜爱度高。"
        ],
        "规则遵守": []
    },
    "draft_with_memory": {
        "行程骨架": [
            "第一天：上午从出发地抵达上海，入住迪士尼附近酒店，下午前往上海迪士尼小镇免费游玩、熟悉氛围，晚上在酒店休息养足体力。",
            "第二天：一早入园畅玩迪士尼乐园，重点体验适合5岁女儿的项目（如小飞侠天空奇遇、小熊维尼历险记、巴斯光年星际营救等），中午在园内用餐并观看花车巡游，下午继续游玩或观看剧场表演（如冰雪奇缘：欢唱盛会），傍晚离园返程。"
        ],
        "输出附带": [
            "建议提前下载上海迪士尼度假区官方APP，实时查看排队时间和领取预约等候卡；",
            "带5岁孩子务必携带儿童推车（或园区可租借），备好防晒、零食和水杯；",
            "迪士尼限流时建议购买早享卡或尊享卡，减少排队等待对孩子耐心的影响；",
            "第二天门票需提前在官方渠道预约购买，避免现场无票；",
            "如果时间充裕，可考虑第三天上午再安排一次迪士尼半日游，但两日行程中第二天已能覆盖大部分亲子项目。"
        ],
        "规则遵守": []
    }
}

```

七、工程边界：谁负责什么，谁不负责什么

角色	负责什么	不负责什么
Session	当前会话历史、窗口裁剪	跨任务的长期记忆治理
文件记忆库	raw、consolidated、semantic、task_brief 等中间产物	工程级权限、索引、并发和查询优化
Workspace	综合案例里的工程化持久化、scope 隔离、ContextPackage	四层记忆理论本身、判定某条记忆一定为真
向量库扩展	语义近义召回加速	主事实存储、证据链、权限
心跳整理器	定时触发抽取、合并、晋升、checkpoint	自己决定记忆理论和业务规则
模型	抽取、同义判断、摘要、表达	独立完成记忆治理

边界分开之后，整套系统变成一组可替换的模块：将来增加 Workspace 或向量库不影响文件产物格式；换抽取模型不影响 checkpoint 和证据链；调整晋升门槛不影响任务接口；加安全策略不用重写业务 Agent。

总结

1. **记忆是治理问题**：RAG 只覆盖召回；写入、修正、遗忘这三个动作决定"这条记录未来该不该影响行为"；
2. **Hermes 给了一个可落地参照**：稳定记忆小而常驻，历史会话完整可查，外部 Memory Provider 只做附加增强；本文用文件产物把这套分工拆成四层；
3. **先用文件系统跑通记忆闭环**：raw 只追加、consolidated 带证据、semantic 少而稳定，每层治理方式不同；Workspace 是工程化后端，向量库只是可重建的召回副本；
4. **抽取靠模型，治理靠代码，验收看行为**：理解自然语言的判断交给模型；可核对的结构化判断留在代码；最终看同一个任务在带记忆和不带记忆时有没有可观察的行为差异。

资料来源

- Hermes Agent Persistent Memory: <https://hermes-agent.nousresearch.com/docs/user-guide/features/memory>

- Hermes Agent Memory Providers: <https://hermes-agent.nousresearch.com/docs/user-guide/features/memory-providers>
- Cognitive Architectures for Language Agents (CoALA): <https://arxiv.org/abs/2309.02427>
- Generative Agents: Interactive Simulacra of Human Behavior: <https://arxiv.org/abs/2304.03442>
- Mind Your HEARTBEAT! Claw Background Execution Inherently Enables Silent Memory Pollution: <https://arxiv.org/abs/2603.23064>
- Chroma Docs: <https://docs.trychroma.com/docs/overview/introduction>
- LanceDB Hybrid Search: <https://docs.lancedb.com/search/hybrid-search>